A Code Rate-Compatible High-Throughput Hardware Implementation Scheme for QKD Information Reconciliation

Ming Zhu, Ke Cui, Simeng Li, Lei Kong, Shibiao Tang, and Jian Sun

***Abstract*—For high-speed quantum key distribution (QKD) post processing, information reconciliation is the most computational step, which usually acts as the system speed bottleneck. Reconciliation efficiency and operation throughput are the two most important performance parameters, but they are often compromised to each other in actual realization due to the available hardware resources. Another characteristic of the information reconciliation is that its channel is time-varying, and in order to guarantee high efficiency, some rate-compatible error correction scheme should be adopted. To cope with the above problems, this paper proposes a rate-compatible high-efficiency reconciliation algorithm based on quasi-cyclic (QC) low-density parity-check (LDPC) codes and puncturing algorithms. On the other hand, in order to obtain high throughput while maintaining low implementation complexity, the normalized min-sum algorithm (NMSA) is realized and optimized by using the fast prototyping tool of high-level synthesis (HLS). The proposed information reconciliation module was implemented on the Xilinx Zynq Ultrascale+ ZCU102 development board. Experimental results show that the maximum throughput rate of the implemented reconciliation module in this paper can reach 136 Mbps, while the efficiency factor is kept lower than 1.32 across the error rate range of 1.7%∼10.6% under the remaining frame error rate level of 10^{-3}. The comprehensive good performance obtained by our design can strongly support the development of modern high-speed QKD systems.**

***Index Terms*—FPGA, HLS, information reconciliation, LDPC, puncturing, QKD.**

I. Introduction

QUANTUM key distribution (QKD) [1] allows two legal parties, namely Alice and Bob, to achieve absolutely secure shared keys even when the eavesdropper can perform arbitrary operations on the channel. The security of the QKD system is based on the quantum mechanics, and its security cannot be diminished by the computation power [2], [3].

Manuscript received October 13, 2021; revised January 23, 2022; accepted February 3, 2022. Date of publication February 8, 2022; date of current version June 16, 2022. This work was supported by the Foundation Enhancement Program of Science and Technology Commission under Grant 2019-JCJQ-JJ-356. *(Corresponding author: Ke Cui.)*

Ming Zhu, Ke Cui, Simeng Li, and Lei Kong are with the School of Electronic and Optical Engineering, Nanjing University of Science and Technology, Nanjing 210094, China (e-mail: njustcuik@njust.edu.cn).

Shibiao Tang and Jian Sun are with the QuantumCtek Company, Ltd., Hefei, Anhui 230088, China.

Color versions of one or more figures in this article are available at https://doi.org/10.1109/JLT.2022.3149567.

Digital Object Identifier 10.1109/JLT.2022.3149567

A typical QKD post-processing [4] mainly includes four steps: (1) basis sifting (2) identity authentication (3) information reconciliation and (4) privacy amplification. The information reconciliation [5]–[9] is needed due to the inconsistent keys between Alice and Bob, which is caused by the influence of channel noise or eavesdropper's monitoring. The information reconciliation is the most computational unit of the post processing, which usually acts as the speed bottleneck in achieving high-throughput post processing capability. Nowadays, by integrating ultra-high repetition lasers and superconductive single-photon detectors into QKD system [10], higher key generation rate is putting forward tenser requirements on the QKD post processing [11], which stimulates the development of high throughput information reconciliation implementations.

The first proposed information reconciliation scheme for QKD system is the BBBSS [12] protocol, which uses a single-bit parity check code as the error correction code. The protocol is simple and intuitive but makes itself hard to achieve high reconciliation efficiency. Then the Cascade protocol is proposed [13], [14] which is widely adopted by many practical QKD systems due to its high efficiency with the reconciliation efficiency factor f being 1.1∼1.3 (the definition of f can be found in Section II). But the Cascade requires multiple interactions during error correction, which greatly limits the reachable implementation speed especially for the QKD systems with high network latency. The Winnow protocol [15] was proposed to overcome the multiple interaction problem, which can obtain higher throughput, but its efficiency performance is not as good as the Cascade with f being about 1.5. In recent years, the LDPC code [16] begins to be adopted in the QKD system due to its excellent performance [17], [18], including the approaching-the-Shannon-limit code capacity, one-time interaction, and high-parallelization-potential decoding algorithm. For practical QKD system, the channel bit error rate may change within a relatively large range, but the fixed rate LDPC code can only maintain high reconciliation efficiency in a narrow bit error rate interval. In order to be adapted to the channel error rate variance, it is possible to include several rates of LDPC code by integrating multiple LDPC matrixes, but this calls for very strict and challenging requirements of the high-performance LDPC matrix designs (given the particular requirement of the LDPC structure such as quasi cyclic (QC) attribute in this paper). Ref. [5] is the pioneering work that adopts the code rate adaptive LDPC codes to the information reconciliation making the reconciliation efficiency

very close to 1. Ref. [6] further to implement the algorithm on GPU which achieve the high throughput of tens of megabytes. In the recent work of Ref. [9], the authors optimized the variable step sizes in the information reconciliation and further improved the computation speed. But the adopted rate adjusting method in the above works was simple, high efficiency could not be guaranteed until long code length was applied, which is very detrimental to hardware implementation.

In this paper, by introducing the dedicated puncturing algorithm [19] to dynamically adjust the LDPC code rate to be adapted to the wide channel error rate range, both high reconciliation efficiency and low implementation complexity can be realized.

Among various hardware platforms, the field programmable gate array (FPGA) holding massive logic elements, rich DSP units and large on-chip RAM storage, has the potential to realize high-throughput QKD post processing [20]–[22]. The traditional FPGA-based hardware design requires the designer to write the register transfer level (RTL) by hand, which is a tedious and time-consuming task. This brings great burden to the designers when frequent system updates or modifications are needed. But the high-level synthesis (HLS) tool emerging in recent years provides an efficient and rapid development method which can accelerate the process of hardware implementation [23], [24].

In this paper, aiming to obtain a information reconciliation module with both high efficiency and high throughput while maintaining low design complexity, the QC-LDPC code utilizing the normalized minimum sum algorithm (NMSA) is adopted. The reconciliation protocol is combined with the puncturing technique to be adapted to large channel error rate range. The protocol is written in C language and realized by Vivado HLS tool which makes the design process rather easy. The optimization technique of the HLS to obtain high throughput was specifically investigated in this paper. The proposed information reconciliation module was implemented on the Zynq Ultrascale+ ZCU102 development board. Experimental results show that the maximum throughput rate of the realized module can reach 136 Mbps, while the reconciliation efficiency factor is kept lower than 1.32 across the error rate range of 1.7%∼10.6% under the remaining frame error rate level of 10^{-3}. The comprehensive good performance obtained by our design can strongly support the development of modern high-speed QKD systems.

II. The QKD Post-Processing and Information Reconciliation

The QKD protocol is composed of the quantum channel transmission and the post processing. The role of the first part is to transmit qubits between Alice and Bob. The role of the second part is to extract secure keys from the transferred qubits. The post-processing flow is shown in Fig. 1, which mainly includes basis sifting, identity authentication, information reconciliation, and privacy amplification. During the basis sifting, Alice announces the used basis to Bob and Bob keeps the corresponding qubits which are measured using the same basis as Alice. Identity authentication is integrated to prevent the man-in-the-middle attack. To eliminate the mismatch of the raw keys between Alice and Bob, the information reconciliation is required to obtain the consistent keys shared at both sides. Privacy amplification is the hashing operation which distills absolutely secure keys from the error free bit keys.

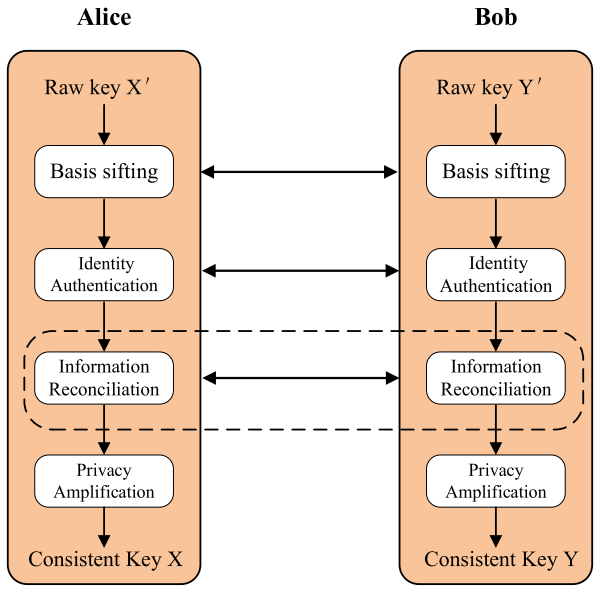

Fig. 1. The post-processing flowchart.

The reconciliation efficiency f is usually chosen as the performance parameter for the information reconciliation [5]. It is defined as the theoretical minimal information limit given by Shannon entropy divided by the actual leaked information during reconciliation process. For the LDPC code, its calculation equation is shown as below:

$$f = \frac{m}{nh(e)} = \frac{1 - R_0}{h(e)} > 1 \tag{1}$$

where m and n are the numbers of the row and column of the LDPC check matrix, $R_0 = 1 - m/n$ is the code rate, e is the channel bit error rate, and $h(e)$ is the Shannon binary entropy:

$$h(e) = -e\log_2 e - (1-e)\log_2(1-e) \tag{2}$$

As reflected by (1), the f is usually larger than 1 for an actual information reconciliation protocol. Therefore, designing some reconciliation protocol with f approaching 1 is the design goal from the perspective of reconciliation efficiency.

III. The Information Reconciliation Protocol Based on the QC-LDPC Code and Puncturing Technique

A. The Information Reconciliation Based on the Puncturing Technique

For the LDPC code with a fixed code rate, it exists the maximal channel error rate that the LDPC code can cope with for the targeted remaining bit error rate (or frame error rate) after error correction [25]. Therefore, to cover a large channel error rate range, multiple LDPC check matrixes should be used which may call for very challenging requirements of the high-performance LDPC matrix designs especially when we want to keep the QC

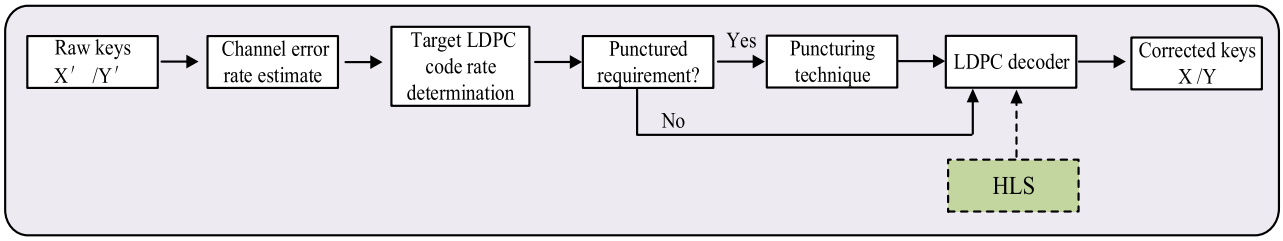

Fig. 2. The flow chart of the information reconciliation with adjustable code rate. X′/ Y′ is the raw key before information reconciliation and X / Y is the corrected one after information reconciliation. The green box represents that the LDPC decoder is designed by using the HLS tool.

style of the LDPC codes. Alternatively, another good solution is to use a single LDPC check matrix combined with the puncturing technique to be adapted to the wide error rate range. [26]–[28].

The puncturing operation is conducted by deleting p parity bits from the mother LDPC code by improving the code rate from $r_0 = k/n$ to $r_1 = k/(n-p)$. Recent studies show that the puncturing scheme using dedicatedly designed puncture bit positions has greatly improved error correction performance than the random puncturing scheme. In this paper, the so-called grouping and sorting (GS) puncturing algorithm proposed by J. Ha [19] is adopted to find the optimal puncturing bit positions for our information reconciliation protocol. The algorithm is mainly divided into two steps: grouping and sorting. By grouping all variable nodes using the concept of recovery tree, the variable nodes that require the same number of recovery steps are placed in the same set. Then the algorithm adjusts the variable nodes with more surviving check nodes to the set front during the sorting step, so that the variable node that requires less recovery steps can be selected firstly. The maximum achievable code rate is expressed by (3) [19]:

$$r_{\max} = \frac{r_0}{1 - \sum_{j=1}^{K} |\mathbf{G}_j|/n} \tag{3}$$

Where r_0 is the code rate of the mother matrix, G_j represents the recoverable node set in the j-th step, $|G_j|$ represents the node number of the set, and K presents the total set number.

According to (3), it can be determined that for the LDPC check matrix with rate of 0.5, the upper code rate limit after puncturing is $r_{\max 1/2} = 0.78$, and for the original code rate of 2/3, the corresponding upper code rate limit can reach $r_{\max 2/3} = 0.89$. So by adopting two mother LDPC check matrixes and combined with the puncturing technique, a wide code rate between 0.5 and 0.89 can be obtained. In this work, by aiming at keeping the reconciliation efficiency lower than about 1.3 and according to our simulation results, the following eight puncturing vectors (the collection of the puncture bit positions) with the rates of r = 0.55, 0.6, 0.7, 0.74, 0.78, 0.81, 0.83, 0.84 are calculated and pre-stored in hardware.

Based on the above puncturing technique, the detailed information reconciliation protocol flow is depicted in Fig. 2.

It mainly contains four steps: 1) the channel error rate is estimated by exchanging some small number of keys between Alice and Bob; 2) according to the initial error rate, the proper target code rate is selected, and whether to integrate the puncturing technique is determined by referring to the error rate versus efficiency curve as shown in Fig. 5 in Section V. The channel bit error rate is divided into ten zones which are represented by ten curves in Fig. 5, and the labeled code rate of the curve in the corresponding error rate zone is chosen as the target for a given channel error rate; 3) if the puncturing technique is required, Bob sets the bit information to 0 according to the positions recorded in the puncture vector; and 4) the LDPC decoder based on the HLS is operated (see the next subsection for details) to correct errors between Alice and Bob.

B. The LDPC Decoder

By balancing the implementation complexity and performance, the QC-LDPC is chosen as the check matrix. The QC-LDPC code has regular structural characteristic to make itself friendly to hardware implementation. Its check matrix H is composed of multiple cyclically shifted sub-matrices. The sub-matrix is described by the base matrix $\mathrm{H_b}$ whose dimension is m^*n. Each element of the $\mathrm{H_b}$ is a sub-matrix with size of z^*z, so the total size of the QC-LDPC is mz^*nz. Each non-negative element of $\mathrm{H_b}$ represents a circulant matrix of the unit matrix, and the value represents the cyclically right shift number. The IEEE 802.16E standard LDPC codes [29] with rates of 1/2 and 2/3A are chosen as the mother matrix. As for the decoding algorithm, the NMSA is adopted considering its hardware-friendly implementation capability and small performance degrade [30] compared with the classical logarithmic domain belief propagation (BP) decoding algorithm [31].

The NMSA decoding process contains five steps: initialization, check node update, variable node update, posterior LLR update and hard decision. The specific NMSA decoding flow is as following:

1) Initialization: The prior logarithmic-likelihood ratio (LLR) of each variable node is calculated according to (4),

$$L^0(q_{ij}) = L(p_i) = \log_2 \frac{P_i^0}{P_i^1} \tag{4}$$

where P_i^0/P_i^1 is the probability that the i-th bit is 0/1, p_i is the i-th priori LLR, and q_{ij} represents that the information is passed from the i-variable node to the j-th check node.

2) Check Node Update: The check-to-variable information $L^k(r_{ij})$ is calculated as:

$$\begin{aligned} L^k(r_{ij}) = (1-2S_j)\prod\nolimits_{i' \in R(j)/i} \mathrm{sgn}\left(L^{k-1}(q_{i'j})\right) \\ \times \min\left|L^{k-1}(q_{i'j})\right| \times \alpha \end{aligned} \tag{5}$$

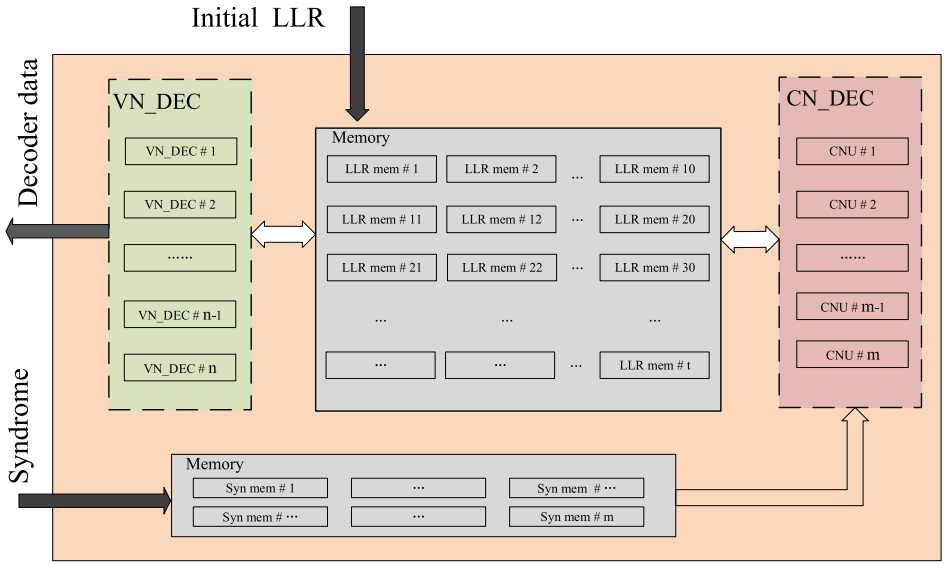

Fig. 3. The LDPC decoder hardware architecture. The CN_DEC block consists of *m* CNU units. The VN_DEC block consists of *n* VN_DEC units. The LLR mem stores the node messages during the decoding and the Syn mem stores the syndromes. The number after the symbol # denotes the parallel unit index.

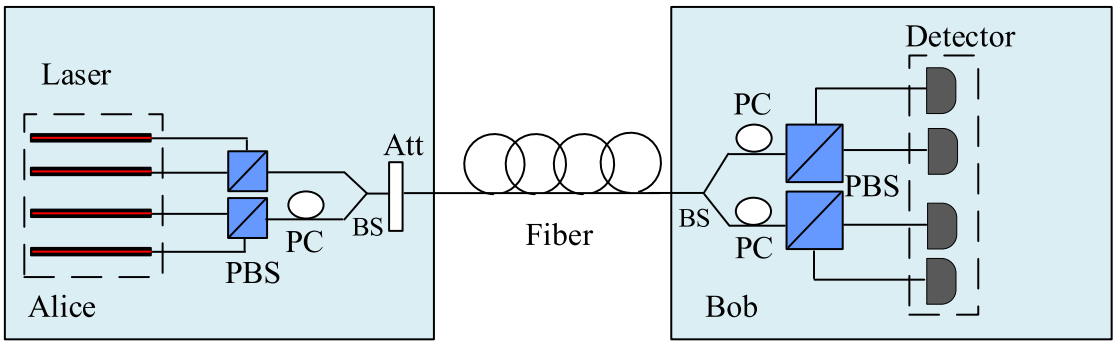

Fig. 4. The QKD system structure. BS: beam splitter; PBS: polarizing beam splitter; PC: polarization controller; Att: attenuator.

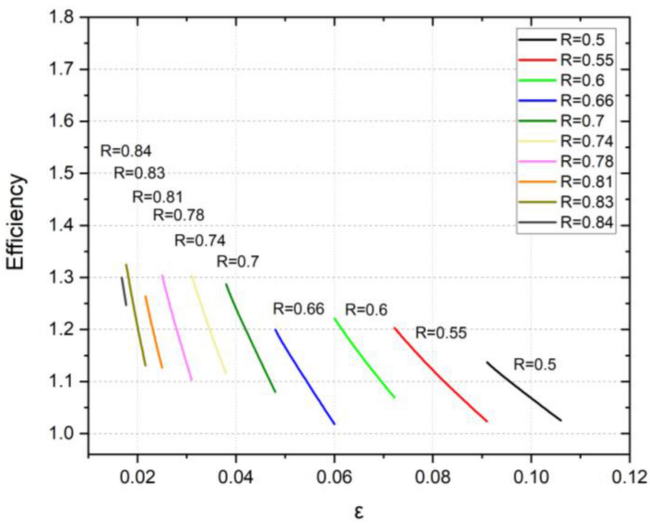

Fig. 5. The reconciliation efficiency with code rates of 0.5, 0.55, 0.6, 0.66, 0.7, 0.74, 0.78, 0.81, 0.83, and 0.84.

Where S_j which is the *j*-th bit of Alice's syndrome S, $i' \in R(j)/i$ represents any neighboring variable node of the *j*-th check node except *i* according to H, α is the normalization factor and k represents the k-th iteration.

3) Variable Node Update: The variable-to-check information $L^k(q_{ij})$ is calculated as:

$$L^k(q_{ij}) = L(p_i) + \sum_{j' \in C(i)/j} L^k(q_{ij'}) \tag{6}$$

Where $j' \in C(i)/j$ represents any neighboring check node of the *i*-th variable node except *j* according to H.

4) Posterior LLR Update: The posterior reliability value, referred to as soft output of the *i*-th variable node, is given by:

$$L^k(q_i) = L(p_i) + \sum_{j \in C(i)} L^k(r_{ij}) \tag{7}$$

Where $j \in C(i)$ represents any neighboring check node of *i* according to H.

TABLE I
HARDWARE RESOURCE CONSUMPTION

Hierarchy				
	Puncturing module	LDPC decoder		Total
		LDPC 1/2	LDPC 2/3A	
LUTs	228	13980	12553	26761
FFs	129	7867	8728	16724
BRAMs	4	38	40	82

5) Hard Decision:

$$\begin{cases} Y_i = 0, & \text{if } L^k(q_i) \geq 0 \\ Y_i = 1, & \text{if } L^k(q_i) < 0 \end{cases} \tag{8}$$

If the decoding result Y satisfies the condition $YH^T = S$, the decoder terminates by outputting the correctly decoded code. Otherwise, the decoder jumps to step 2 to launch a new loop until the maximum iteration number is reached.

IV. THE HLS-BASED HIGH THROUGHPUT LDPC DECODER IMPLEMENTATION

Traditional FPGA design flow requires the developers to write RTL to implement the corresponding algorithm. Such development method requires the designer to have rich development experience and cost long time for complex projects. In this paper, we instead use the HLS tool, whose development language is C/C++, to implement the propose information reconciliation protocol to speed up the development time and keep the future updates flexible and much easier. The HLS tool can automatically analyze the high-level description and map the high-level language to the corresponding register level circuit. It also provides the designers with powerful optimization instructions, such as pipeline, loop unroll, array partition to improve the throughput of the automatically generated circuit [23]–[24].

As shown in Algorithm 1 and 2, the algorithm flow is written in C language and the adopted acceleration directives provided by the HLS tool are listed in the algorithms. The Algorithm 1 flow mainly contains two processes. Process 1 initializes all the needed variables by the algorithm, and Process 2 conducts the decoding task. It contains I_{max} iterations whose value is determined by simulation to be 20, and this can make the remaining frame error rate smaller than 10^{-3}. Process 2.1 performs the check node update (abbreviated by CN_DEC). Process 2.2 performs the variable node update, posterior probability update and hard decision (abbreviated by VN_DEC), and its function is described by Algorithm 2.

According to the code structure (lines 13∼17 of Algorithm 1), the VN_DEC contains *n* submodules (labelled by VN_DEC #k (k = 1, 2…, n)) which will be conducted in parallel. The specific function of the VN_DEC is illustrated in Algorithm 2. The detailed optimization strategy of the hardware architecture is described in the following.

Algorithm 1: The NMSA flow.

```
1 Process 1 : Initialization (Eq. 4)
2 Process 2 : Iterative decoding
3 For iteration = 1,2 …, I_max
4   Process 2.1: check node update (Eq. 5)
5   For z_i = 1,2, …, z
6   #pragma HLS pipeline
7    For j = 1, …,m
8    #pragma HLS unroll
9    Execute CN_DEC update
10   End
11 End
12   Process 2.2: variable node update, posterior
     probability update and hard decision (Eq. 6-8)
13   Execute VN_DEC#1 update
14   Execute VN_DEC#2 update
15   ……
16   Execute VN_DEC#n-1 update
17   Execute VN_DEC#n update
18 End
```

Algorithm 2: The VN_DEC #k (k = 1, 2…, n) Update.

```
1 For z_i = 1, 2, …, z
2   #pragma HLS pipeline
3   Execute VN update (Eq. 6)
4   Execute posterior probability update (Eq. 7)
5   Execute hard decision (Eq. 8)
6 End
```

A. Storage Optimization Strategy

Assuming that t equals $\sum_{i=1}^{m} d_i$, where d_i represents the *i*-th row degree. To accelerate the check node update, i.e., the parallel update of *m* CNU modules, the required data amount of t should be acquired in one clock cycle, otherwise the memory access will become the throughput bottleneck. Therefore, the memory that stores the temporal node messages should be divided into t independent parts. In this way, the check node information update can be finished in just one clock cycle by avoiding memory access conflicts. In the HLS tool, array partition directive is used to realize this goal. The implemented memory organization is shown in Fig. 3.

B. Parallel Update Optimization

According to the structure of the QC-LDPC, the parallelism of the variable node update is designed to be *n*. To further increase the throughput, the check node update should also be optimized to operate in parallel. The HLS tool uses the unroll directives [23] to easily realize the goal (line 8 of Algorithm 1) which can improve the parallelism to *m*. Therefore, it just takes 2^*z clock cycles to complete one decoding iteration. The temporal information during the nodes update are stored in the distributed t RAMs to accelerate the data acquisition as illustrated in section A. The realized computation hardware architecture is shown in Fig. 3.

TABLE II
PERFORMANCE COMPARISONS WITH RELATED WORKS

	Works					
	[4]	[5]	[6]	[9]	[35]	This work
Platform	FPGA-Virtex7	-	GPU	-	FPGA-Kintex UltraScale	FPGA-Zynq UltraScale+
N	262144	2000	1944	64800	1944	1536
Type	RTL	-	-	-	RTL	HLS
$f_{clk}(Mhz)$	-	-	-	-	25	250
BRAMs	1745	-	-	-	210	82
LUTs	146542	-	-	-	102 622	26761
FFs	95066	-	-	-	49 909	16724
I_{max}	30	21	-	-	10	20
FER	10^{-2}	10^{-3}~10^{-2}	10^{-3}~10^{-1}	-	10^{-2}	10^{-3}
$\Gamma(Mbps)$	100.90	-	10~50	-	27.85	136
Error range	-	[3%,9%]	-	[1%,10%]	-	[1.7%,10.6%]
f	-	1~1.7	-	1-1.8	1.49	1~1.32

C. Wide Pipeline Strategy

The pipeline directive (line 6 of Algorithm 1 and line 2 of Algorithm 2) is another important optimization instruction provided by the HLS tool to improve the decoding throughput by synthesizing the serial execution process in the algorithm flow into a proper pipeline [32]. The initial interval is a measure that indicates how long the loop structure can start the execution of a new transaction after it starts executing the previous event. By using the pipeline direction optimization, the initial interval of each check and variable node update modules can be reduced to 1, which greatly improves the overall throughput.

V. EXPERIMENTAL RESULTS

The proposed information reconciliation protocol is written in C language and compiled by using the HLS tool (version 2018. 3). For the LDPC decoder, the normalization factor α of the 1/2 code rate is chosen to be 0.79, and the one of the 2/3A code rate is chosen to be 0.70 according to our simulation result. The compiled hardware circuit was implemented on the platform of Zynq Ultrascale+ ZCU102 (xczu9eg-ffvb1156-2-i) development board. The obtained reconciliation module was tested by using both the simulation data and the actual data obtained from a practical QKD system with channel error rate from 1.7%∼10.6%. The error rate range is divided to ten intervals of (1.7%∼1.77%], (1.77%∼2.16%], (2.16%∼2.5%], (2.5%∼3.1%], (3.1%∼3.8%], (3.8%∼4.8%], (4.8%∼6%], (6%∼7.2%], (7.2%∼9.1%], and (9.1%∼10.6%] corresponding to the ten punctured LDPC code rates. For the simulation data, totally 100,000 frame codes are generated according to the binary symmetric channel (BSC) model, by assigning each error rate interval 10,000 randomly generated frame codes. For the QKD system, the raw keys after basis sifting from Alice and Bob are collected during the running period of two days. This provides enough data amount to pick out and assign each channel error rate interval 10,000 frame codes. All the results shown below are obtained by making the remaining frame error rate lower than 10^{-3} after the information reconciliation.

A. The QKD System

Our used QKD system is based on the polarization-encoding BB84 protocol combined with decoy method [33], [34]. The system structure is shown in Fig. 4. Alice has four laser sources with the repetition rate of 320 MHz to generate the four polarization states via the polarizing beam splitter (PBS) and the polarization controller (PC). The average light intensity is controlled via an attenuator (Att). Each laser can be configured to produce three different light intensity levels representing the signal, decoy and vacuum states respectively. The ratio between the signal, decoy, and vacuum states is 6:1:1 and the mean photon numbers of them are 0.6, 0.2, 0 per pulse. Bob uses the four-channel InGaAs single-photon detector and the PC-based polarization feedback method to detect the qubits. The detection efficiency is about 10%, the dark count is about 10^{-6}, the dead time is 2 μs, and the afterpulse probability is less than 0.5%.

B. Performance Evaluation

To optimize the throughput, besides adopting the circuit architecture optimization illustrated in section IV, fixed data format is chosen for the decoding computation rather than float data format. Fixed data format also helps to save the overall resource cost. According to our simulation result, the 9-bit quantization (with 1 bit of sign, 3 bits of integer and 5 bits of decimal) is selected for our reconciliation module.

The reconciliation efficiency result is shown in Fig. 5. The covered error rate is from 1.7% to 10.6% while keeping the reconciliation efficiency smaller than 1.32. It totally contains ten code rates of 0.5, 0.55, 0.6, 0.66, 0.7, 0.74, 0.78, 0.81, 0.83, 0.84, which is obtained from two QC-LDPC check matrixes by adopting the puncturing technique. The result is pretty good compared with existing reconciliation protocols. The benefits are gained from the dedicated GS puncturing algorithm and the carefully designed codes rate collection. The figure shows

the saw shape. Since for a given code rate, the reconciliation efficiency deteriorates with the reduced channel error rate. After being switched to the next higher code rate, the reconciliation efficiency can be improved apparently.

The throughput Γ of the proposed reconciliation module is expressed by the following formula:

$$\Gamma = \frac{N \times f_{clk}}{(2 \times Z + \theta) \times I_{\max}} \quad (Mbps) \tag{9}$$

where f_{clk} represents the operating frequency, θ represents the overall initialization interval (equal to 13), and $I_{\max}$ represents the max number of iterations (equal to 20). The f_{clk} is determined by the Vivado synthesis report and chosen to be 250 Mhz. The code length N is equal to 1536, and the sub-matrix size Z is equal to 64. By taking the design parameters into the formula, the decoder throughput rate is calculated to be 136 Mbps.

C. Hardware Resource Consumption and Performance Comparison

The hardware hierarchy is mainly composed of the puncturing module and the LDPC decoder module. The LDPC decoder module integrates 1/2 and 2/ 3A code rate decoders. The resource consumption of the corresponding decoding module is shown in TABLE I.

To give fair performance evaluation of this work, we compare it with recent related works and the compassion results are listed in TABLE II. Since our work uses short-length QC LDPC and dedicated puncturing algorithm, the obtained results completely outperform other works in the perspectives of reconciliation efficiency, throughput and resource cost. It should be noted that the resource cost can be further reduced if the tedious VHDL design process is adopted, but the design time will increase accordingly.

VI. Conclusion

In summary, this paper proposes a high efficiency and high throughput information reconciliation protocol based on the QC-LDPC utilizing the NMSA and puncturing technique. The design complexity is also kept low and the future update is made flexible by using the HLS tool. The prototype module circuit was implemented on the Zynq Ultrascale+ ZCU102 development board. Experimental results show that the maximum throughput of 136 Mbps and the efficiency factor lower than 1.32 across the error rate range of 1.7%$\sim$10.6% can be obtained. The comprehensive good performance obtained by our design demonstrates itself a very powerful candidate for high-speed QKD post-processing tasks.

References

[1] J. Wang *et al.*, "Direct and full-scale experimental verifications towards ground–satellite quantum key distribution," *Nature Photon.*, vol. 7, no. 5, pp. 387–393, Apr. 2013.

[2] D. Mayers, "Unconditional security in quantum cryptography," *J. Assoc. Comp. Mach.*, vol. 48, no. 3, pp. 351–406, May 2001.

[3] V. Scarani, H. Bechmann-Pasquinucci, N. J. Cerf, M. Dusek, N. Lutkenhaus, and M. Peev, "The security of practical quantum key distribution," *Rev. Modern Phys.*, vol. 81, no. 3, pp. 1301–1350, Sep. 2009.

[4] S. Yang, Z. Lu, and Y. Li, "High-speed post-processing in continuous-variable quantum key distribution based on FPGA implementation," *J. Lightw. Technol.*, vol. 38, no. 15, pp. 3935–3941, Aug. 2020.

[5] J. Martinez-Mateo, D. Elkouss, and V. Martin, "Blind reconciliation," *Quantum Inf. Comput.*, vol. 12, no. 9–10, pp. 0791–0812, Sep. 2012.

[6] J. Martinez-Mateo, D. Elkouss, and V. Martin, "Key reconciliation for high performance quantum key distribution," *Sci. Rep.*, vol. 3, no. 1, pp. 1–6, Apr. 2013.

[7] Q. Li, X. Wen, H. Mao, and X. Wen, "An improved multidimensional reconciliation algorithm for continuous-variable quantum key distribution," *Quantum Inf. Process.*, vol. 18, no. 1, pp. 1–20, 2019.

[8] Z. Bai, S. Yang, and Y. Li, "High-efficiency reconciliation for continuous variable quantum key distribution," *Japanese J. Appl. Phys.*, vol. 56, no. 4, 2017, Art. no. 044401.

[9] Z. Liu, Z. Wu, and A. Huang, "Blind information reconciliation with variable step sizes for quantum key distribution," *Sci. Rep.*, vol. 10, no. 1, pp. 1–8, Jan. 2020.

[10] A. Tanaka, M. Fujiwara, K. Yoshion, and S. Takahashi, "High-speed quantum key distribution system for 1-Mbps real-time key generation," *IEEE J. Quantum Electron.*, vol. 48, no. 4, pp. 542–550, Apr. 2012.

[11] Z. Yuan *et al.*, "10-Mb/s quantum key distribution," *J. Lightw. Technol.*, vol. 36, no. 16, pp. 3427–3433, Jun. 2018.

[12] C. H. Bennett, F. Bessette, G. Brassard, and L. Salvail, "Experimental quantum cryptography," *J. Cryptol.*, vol. 5, no. 1, pp. 3–28, Sep. 1992.

[13] K. Cui, J. Wang, H. Zhang, C. Luo, G. Jin, and T. Chen, "A real-time design based on FPGA for expeditious error reconciliation in QKD system," *IEEE Trans. Inf. Forensics Secur.* , vol. 8, no. 1, pp. 184–190, Jan. 2013.

[14] G. Brassard and L. Salvail, "Secret-key reconciliation by public discussion," in *Proc. Workshop Theory Appl. Cryptogr. Techn.*, Berlin, Heidelberg, 1994, pp. 410–423.

[15] W. T. Buttler, S. K. Lamoreaux, J. R. Torgerson, G. H. Nickel, C. H. Donahue, and C. G. Peterson, "Fast, efficient error reconciliation for quantum cryptography," *Phys. Rev. A*, vol. 67, no. 5, May 2003, Art. no. 36079653.

[16] G. Robert, "Low-density parity-check codes," *IRE Trans. Inf. Theory*, vol. 8, no. 1, pp. 21–28, Jan. 1962.

[17] D. Ahmad, A. C. Carusone, and F. R. Kschischang, "Block-interlaced LDPC decoders with reduced interconnect complexity," *IEEE Trans. Circuits Syst. II: Exp. Briefs*, vol. 55, no. 1, pp. 74–78, Jan. 2008.

[18] Q. Li, S. Ma, H. Mao, and L. Meng, "An FPGA-based communication scheme of classical channel in high-speed QKD system," in *Proc. 10th Int. Conf. Intell. Inf. Hiding Multimedia Signal Process.*, Kitakyushu, Japan, Aug. 2014, pp. 227–230.

[19] J. Ha, J. Kim, D. Klinc, and S. W. McLaughlin, "Rate-compatible punctured low-density parity-check codes with short block lengths," *IEEE Trans. Inf. Theory*, vol. 52, no. 2, pp. 728–738, Feb. 2006.

[20] C. Hui, Y. Wang, and X. Lu, "Implementation of a high throughput LDPC code in FPGA for QKD system," in *Proc. 13th IEEE Int. Conf. Solid-State Integr. Circuit Technol.*, Hangzhou, China, Oct. 2016, pp. 1494–1496.

[21] H.-F. Zhang *et al.*, "A real-time QKD system based on FPGA," *J. Lightw. Technol.*, vol. 30, no. 20, pp. 3226–3234, Oct. 2012.

[22] P. Hailes, L. Xu, R. G. Maunder, B. M. Al-Hashimi, and L. Hanzo, "A survey of FPGA-based LDPC decoders," *IEEE Commun. Surveys Tuts.*, vol. 18, no. 2, pp. 1098–1122, May–Jul. 2016.

[23] G. Choi, K. B. Park, and K. S. Chung, "Optimization of FPGA-based LDPC decoder using high-level synthesis," in *Proc. 4th Int. Conf. Commun. Inf. Process.* New York, NY, USA, Nov. 2018, pp. 256–259.

[24] S. Liu, F. C. Lau, and B. C. Schafer, "Accelerating FPGA prototyping through predictive model-based HLS design space exploration," in *Proc. ACM/IEEE Conf. 56th Des. Automat.*, Las Vegas, USA, Jun. 2019, pp. 1–6.

[25] T. J. Richardson and R. L. Urbanke, "The capacity of low-density parity-check codes under message-passing decoding," *IEEE Trans. Inf. Theory*, vol. 47, no. 2, pp. 599–618, Feb. 2001.

[26] D. Elkouss, J. M. Mateo, and V. Martin, "Untainted puncturing for irregular low density parity-check codes," *IEEE Wireless Commun. Lett.*, vol. 1, no. 6, pp. 585–588, Dec. 2012.

[27] X.-Q. Jiang, P. Huang, D. Huang, D.-K. Lin, and G.-H. Zeng, "Secret information reconciliation based on punctured low-density parity-check codes for continuous-variable quantum key distribution," *Phys. Rev. A*, vol. 95, no. 2, Feb. 2017, Art. no. 022318.

[28] C. Zhou, X. Wang, Z. Zhang, S. Yu, Z. Chen, and H. Guo, "Rate compatible reconciliation for continuous-variable quantum key distribution using raptor-like LDPC codes," *Sci. Chin. Phys. Mechan. Astron.*, vol. 64, Apr. 2021, Art. no. 6.

[29] G. Falcao, L. Sousa, and V. Silva, "Massively LDPC decoding on multicore architectures," *IEEE Trans. Parallel Distrib. Syst.*, vol. 22, no. 2, pp. 309–322, Feb. 2011.
[30] Z. Zhang, L. Dolecek, B. Nikolic, V. Anantharam, and M. J. Wainwright, "Design of LDPC decoders for improved low error rate performance: Quantization and algorithm choices," *IEEE Trans. Commun.*, vol. 57, no. 11, pp. 3258–3268, Nov. 2009.
[31] M. Tavares, E. Matus, S. Kunze, and G. Fettweis, "A dual-core programmable decoder for LDPC convolutional codes," in *Proc. IEEE Int. Symp. Circuits Syst.*, Seattle, WA, USA, May. 2008, pp. 532–535.
[32] J. Andrade, F. Pratas, G. Falcao, V. Silva, and L. Sousa, "Combining flexibility with low power: Dataflow and wide-pipeline LDPC decoding engines in the Gbit/s era," in *Proc. ASAP Conf.*, Zurich, Switzerland, Jun. 2014, pp. 1–6.
[33] T.-Y. Chen *et al.*, "Implementation of a 46-node quantum metropolitan area network," *NPJ Quantum Inf.*, vol. 7, no. 1, Sep. 2021, Art. no. 134.
[34] T.-Y. Chen *et al.*, "Metropolitan all-pass and inter-city quantum communication network," *Opt. Exp.*, vol. 18, no. 16, pp. 27217–27225, Aug. 2010.
[35] C. Kun, S. Qi, S.-K. Liao, and C.-Z. Peng, "Implementation of encoder and decoder for LDPC codes based on FPGA," *J. Syst. Eng. Electron.*, vol. 30, no. 4, pp. 642–650, Aug. 2019.

Ming Zhu is currently working toward the master's degree with the Nanjing University of Science and Technology, Nanjing, China. His research interests mainly include LDPC decoder and QKD post-processing.

Ke Cui received the B.S. and Ph.D. degrees from the University of Science and Technology of China, Hefei, China, in 2004 and 2014, respectively. He is currently an Associate Professor with the Nanjing University of Science and Technology, Nanjing, China. In 2019, he was a Visiting Scholar with Stanford University, Stanford, CA, USA. His research interests include real time signal processing and high-performance computing based on FPGA, technologies of optical fiber sensors.

Simeng Li is currently working toward the master's degree with the Nanjing University of Science and Technology, Nanjing, China. His research interests include HMM model and QKD post-processing.

Lei Kong received the B.S. degrees from the Harbin Engineering University of China, Harbin, China, in 2015. He is currently working toward the M.S. degree with the Nanjing University of Science and Technology, Nanjing, China. His research interests include optical fiber interference system and deep learning.

Shibiao Tang received the B.S. and Ph.D. degrees from the University of Science and Technology of China, Hefei, China, in 2004 and 2009, respectively. He is currently a Senior Engineer with QuantumCtek Co. Ltd. His research interests include QKD system and high-speed electronics.

Jian Sun received the B.S. and Ph.D. degrees from the University of Science and Technology of China, Hefei, China, in 2004 and 2012, respectively. He is currently an Engineer with QuantumCtek Co. Ltd. His research interests include QKD system and physical electronics.